For the custom scheduling algorithm part of lab 3, I chose to implement a version of Linux's Completely Fair Scheduler (CFS) policy in xv6. I implemented a *vruntime* field in each proc struct that gets increased following each interval of execution. The formula to calculate the increase is $\Delta vruntime = t_{execution} \cdot 1.25^{-min(priority, 10)}$. This process enforces the policy that high priority process will have a larger magnitude for the negative exponent, and thus the $load\_weight$ scalar will diminish the effective execution time. This allows higher priority processes to be treated as if they have not yet had their fair share of execution time. Upon each instance of yielding, triggered by the timer, the *vruntime* will be updated appropriately. I opted not to use a red-black tree in my implementation or any data structure for that matter since the limit of 256 processes at a given time in xv6 makes the Big-O efficiency negligible at best compared to modern operating systems designed for thousands of processes. For this lab, since the *workload.c* test did not run processes of varying priorities, this feature is not explicitly tested, but it is still included so that it can be used for other workloads.

Figure 1 shows the cumulative distribution for latency between the 3 scheduling policies running the workload test running xv6 on qemu with 1 CPU. The majority of processes have very low response times due to permitting I/O bound processes to execute sooner with higher effective priority.

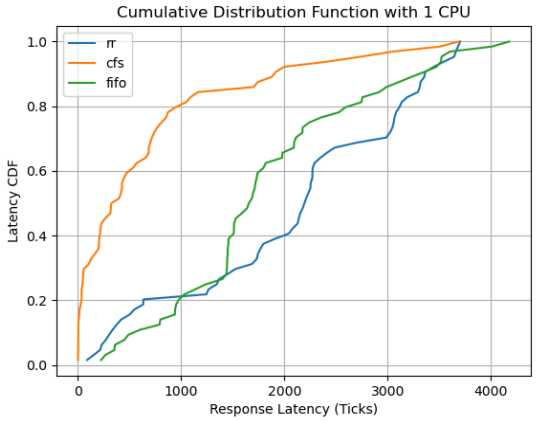

Figure 2 shows the cumulative distribution for latency running the workload on 4 CPUs. This CFS implementation seems to not perform as efficiently compared to the other policies. This is likely due to the fact that less starvation would occur for I/O bound processes with the other two policies on additional available CPUs.